

SecurityNode

Application Note

Controller / Application Node Design

Final Project of Electronic Design

Version 1.1 (schematic-verified) — June 17, 2026

Abstract

This application note documents the **design** of the SecurityNode controller/application node. The node is implemented on the ESP32-C3-WROOM-02-N4 and is responsible for sensor monitoring, alarm control, vision-module coordination, and host communication. Because the project is still at the design stage and no assembled hardware is available, this note focuses on architectural choices, electrical interfaces, requirements and traceability, a formal state machine, the UART protocol, and a theoretical verification plan. Measured bench data such as oscilloscope waveforms, thermal tests, and power-consumption plots are intentionally excluded.

Status and verification note: This is a design-stage, not-yet-fabricated project: no hardware has been built and no firmware has been written. The *pin map and electrical facts* in this note (pinout, power tree, U4/U5 roles, designators) are **verified against the schematic netlist**. All *firmware, communication-protocol, finite-state-machine, and host-tool content* is a *PROPOSED design*, not as-built, and is labelled as such throughout.

Rev.	Date	Change
1.0	—	Initial design-stage application note.
1.1	June 17, 2026	Pinout, power tree, and U4/U5 roles corrected and verified against the schematic netlist; schematic/PCB figures, component-value rationale, key-components table, and references added.

Contents

1	Purpose and scope	3
2	Design assumptions	3
3	Design artifacts	4
4	Requirements	5
5	Traceability matrix	5
6	External interfaces	6
6.1	Pinout	6
6.2	UART interface parameters	7
6.3	Vision approach (open design decision)	8

7	Electrical design summary	8
7.1	Key components	8
7.2	Power architecture	9
7.3	Camera power gate (U4) and UART isolation switch (U5)	9
7.4	Analytical power budget	10
7.5	Output drive	10
7.6	Component value rationale	10
7.7	Schematic preview	11
7.8	PCB design review notes	13
8	State machine	13
8.1	State definitions	14
8.2	State-transition diagram	14
8.3	State behavior table	16
9	UART protocol interface	16
9.1	Frame format	16
9.2	Command set	16
9.3	Response handling	17
9.4	Timeout and retry rules	17
10	Firmware design sketch	17
11	Additional design considerations	18
11.1	Mechanical and environmental	18
11.2	Security considerations	18
12	Theoretical verification plan	18
12.1	Expected behavior and design targets	18
12.2	Analytical verification	19
12.3	Schematic and PCB verification	19
12.4	Excluded verification activities	19
13	References	19
14	Summary	20

1 Purpose and scope

This application note has three objectives:

1. Capture the design intent of the SecurityNode application/controller node.
2. Provide enough detail for a peer review of the firmware architecture, electrical interfaces, and communication protocol before hardware is built.
3. Define the verification activities that can be performed analytically or by inspection, given that no physical prototype exists yet.

The note covers:

- Design assumptions and constraints.
- Design artifacts produced by the project.
- Functional requirements and a traceability matrix.
- External interfaces and a formal pinout.
- Electrical design summary and review notes.
- Finite-state machine with a camera-availability flag.
- UART protocol interface between the ESP32-C3 and the ESP32-CAM.
- Firmware design sketch.
- Theoretical verification plan.

The following topics are **out of scope** until hardware is fabricated: measured waveforms, oscilloscope captures, thermal performance, real power consumption, and PCB bring-up results.

2 Design assumptions

The design described in this note relies on the assumptions listed in Table 1.

Table 1: Design assumptions.

ID	Area	Assumption
A1	Controller	The application node runs on an ESP32-C3-WROOM-02-N4.
A2	Vision	The ESP32-CAM is connected through UART1 on IO0=RX / IO1=TX (auto-isolated by U5, a CAM-presence-gated series switch) at 115200 baud.
A3	Sensors	The door contact and PIR (the PIR is 5 V-powered, its signal reaching the C3 through a clamp/divider network) reach the C3 as logic-level signals; polarity is configurable in firmware.
A4	Power	The board is powered from a 5 V USB source through a PTC fuse (F1); a 5 V rail then feeds, in parallel, the 3.3 V LDO (U1) for the C3+U5 logic and the CAM power gate (U4).
A5	Outputs	The buzzer is driven by a low-side N-channel MOSFET (Q1, DN2302S); the alarm LED (D5) has a series current-limiting resistor (R15, 330 Ω).
A6	Host	The C3 is programmed and debugged over its native USB (USB-Serial/JTAG, IO18/IO19); a separate UART0 debug header (J1, GPIO20/21) is also provided. No USB-UART bridge chip is present.
A7	Algorithm	The ESP32-CAM returns one of the result codes defined in the CAM protocol; the C3 does not run vision inference. The on-device vision approach is a <i>deliberately open design decision</i> (see §6.3).
A8	Backup	No battery backup is provided; the system operates from USB power only.

3 Design artifacts

The following artifacts are part of the SecurityNode design baseline and are referenced throughout this note:

- `docs/system/system_overview.md` — System description and block diagram.
- `docs/firmware/firmware_architecture.md` — Firmware structure and build options.
- `docs/firmware/cam_protocol.md` — Binary UART protocol specification.
- `docs/hardware/` — Hardware interface notes and design rationale.
- `hardware/altium/` — *Planned* Altium schematic and PCB source files (directory is currently a placeholder; the schematic netlist and BOM are the present source of truth).
- `firmware/`, `software/host/` — *Planned* firmware and host-tool source (directories are currently placeholders; no code written yet).
- `assets/Schematic Blocks/` — Schematic-block previews.

- `assets/pcb.pdf` — PCB fabrication preview.
- This document — Application Note for the controller node.

4 Requirements

Table 2: Controller/Application Node functional requirements.

ID	Requirement	Rationale	Priority	Verification
R1	Monitor door contact continuously.	Detect unauthorized entry.	High	Inspection / Analysis
R2	Monitor PIR motion continuously.	Detect movement inside the protected area.	High	Inspection / Analysis
R3	Request visual verification from the ESP32-CAM on a sensor event.	Gate the alarm on visual evidence: catch masked / covered-face intruders while suppressing benign triggers.	High	Protocol review
R4	Activate the local alarm when a suspicious condition is detected.	Deter intruders and alert occupants.	High	State-machine review
R5	Operate as a basic alarm when the vision module is unavailable.	Maintain security even if the CAM fails.	High	State-machine review
R6	Provide a debug/configuration channel: native USB (USB-Serial/JTAG) plus a UART0 header (J1).	Allow setup, diagnostics, and logging.	Medium	Interface test
R7	Recover from transient UART errors through retry and timeout.	Improve communication robustness.	Medium	Protocol review
R8	Operate from a 5 V USB supply within standard current limits.	Simplify deployment and avoid overload.	Medium	Power budget analysis

5 Traceability matrix

Table 3 maps each requirement to the sections of this note that address it and the planned verification method.

Table 3: Requirements traceability matrix.

Req.	Addressed in	Verification activity
R1	§6.1, §8, §10	GPIO input review; state coverage analysis.
R2	§6.1, §8, §10	GPIO input review; state coverage analysis.
R3	§9, §8	Protocol inspection; state-transition review.
R4	§8, §7	Alarm-path review; output-drive analysis.
R5	§8 (cam_available flag)	State-machine review; CAM-failure scenario.
R6	§6.1, §9	Native-USB and UART0-header interface review.
R7	§9 (timeout/retry rules)	Protocol robustness analysis.
R8	§7	Analytical power budget.

6 External interfaces

Figure 1 shows the controller node and its surrounding interfaces.

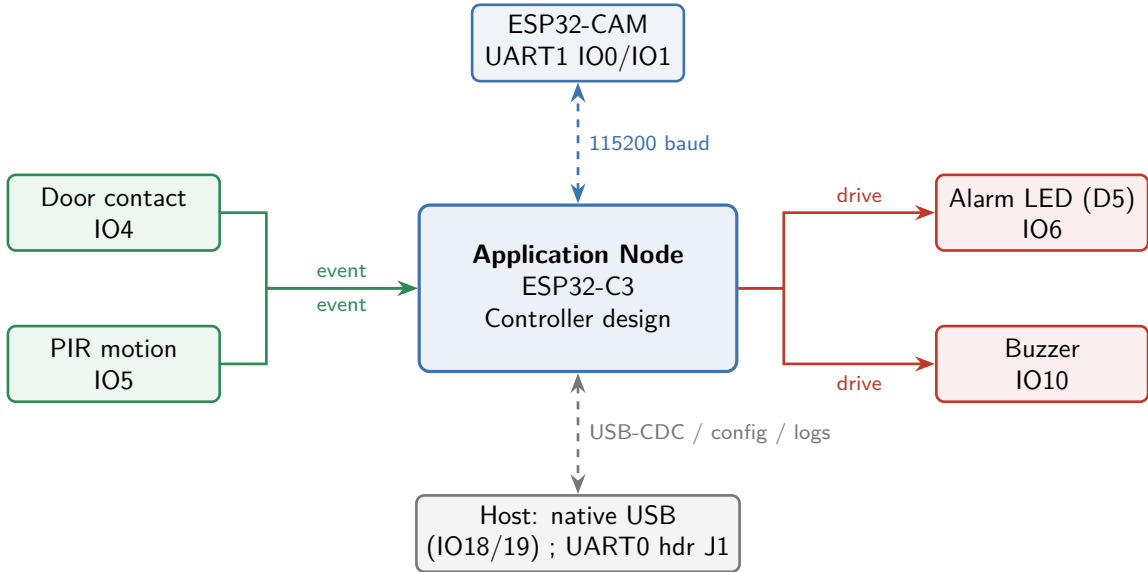


Figure 1: Application Node block diagram and external interfaces.

6.1 Pinout

Table 4 lists the controller pins used by the application node.

Table 4: Application Node pin assignment.

Signal	Pin	Dir.	Function	Electrical notes
DOOR_IN	IO4	Input	Door contact state	Logic-level input; clamp/RC debounce network on board.
PIR_IN	IO5	Input	PIR motion detect	PIR powered from 5 V; signal reaches IO5 via clamp/divider; edge or polled.
ALARM_LED	IO6	Output	Alarm LED drive	Drives D5 via R15 (330 Ω) series resistor.
BUZZER_DRV	IO10	Output	Buzzer MOSFET gate	Drives Q1 (DN2302S) low-side switch via R12; 5 V buzzer load (LS1).
CAM_TX	IO1	Output	UART1 TX to ESP32-CAM	3.3 V, 115200 baud, via R21 + U5; no flow control.
CAM_RX	IO0	Input	UART1 RX from ESP32-CAM	3.3 V, 115200 baud, via R20 + U5; no flow control.
USB_TX	IO21	Output	UART0 TX (debug header)	Wired direct to J1 (pin 2); not through U5.
USB_RX	IO20	Input	UART0 RX (debug header)	Wired direct to J1 (pin 3); not through U5.
USB_D	IO18/19	I/O	Native USB (D ⁻ /D ⁺)	USB-Serial/JTAG to J2 via R3/R4 (22 Ω) + U2 ESD; programming/CDC.
3V3	—	Power	Logic supply rail	AP2112K-3.3 LDO (U1) output; feeds C3 + U5 only.
VBUS	—	Power	5 V USB input	Over-current protected by PTC fuse F1; TPS2553 (U4) gates CAM_5V only.
GND	—	Power	Common ground	Single-point/star grounding per PCB review.

6.2 UART interface parameters

Table 5: UART parameters for the ESP32-CAM link.

Parameter	Value
Controller port	UART1 (IO0=RX / IO1=TX, via U5 isolation switch)
Baud rate	115200
Data bits	8
Parity	None
Stop bits	1
Flow control	None
Frame format	Binary header/length/ID/payload/checksum

6.3 Vision approach (open design decision)

This is a deliberately open, design-stage decision. The application node treats the ESP32-CAM as a black box that returns a face-coverage result code over UART (see the protocol in §9), but *how* that result is computed is not yet settled. A key constraint is that a full object-detection model (e.g. YOLO) does **not** fit on the ESP32-CAM: the OV2640-based module is heavily constrained in RAM, flash, and compute. Three candidate approaches are under evaluation, each with different trade-offs:

(a) Tiny on-device classifier.

A small quantized classifier (ESP-DL or TFLite-Micro) running on the ESP32-CAM, performing a coarse *face-present* / *face-covered* judgement on small, downscaled frames. *Trade-off*: fits entirely on-device and keeps the architecture single-tier, but offers only modest accuracy and limited robustness to pose, lighting, and occlusion variety.

(b) Off-device inference (two-tier).

The CAM captures a frame and streams it over Wi-Fi to a host or edge device running a full model (e.g. YOLO or a stronger classifier), which returns the verdict. *Trade-off*: full accuracy and model flexibility, but requires an always-available host and changes the system from one self-contained board into a **two-tier** architecture (latency, networking, and availability dependencies).

(c) Built-in face detection as a coverage proxy.

Use only ESP-DL’s built-in face-detection primitive and treat “no detectable face” as a proxy for a covered face. *Trade-off*: simplest to implement and cheapest on-device, but the weakest semantically — it cannot reliably distinguish a *covered face* from *no person present*, which risks both false alarms and missed detections.

The choice among (a), (b), and (c) is intentionally left open at this stage; it should be resolved with an explicit memory/latency budget before firmware is written. Assumption A7 references this open decision rather than asserting any particular on-device method as settled.

7 Electrical design summary

7.1 Key components

Table 6 lists the principal devices and their roles; the full passive network and exact part numbers are in the schematic and the project BOM (`assets/SecurityNode.BomDoc`).

Table 6: Key components (designator $\rightarrow$ part $\rightarrow$ role).

Ref.	Part	Role
U1	AP2112K-3.3TRG1	3.3 V / 600 mA LDO (logic rail).
U2	USBL6-2P6	USB D+/D- ESD protection.
U3	ESP32-C3-WROOM-02-N4	Controller / application node.
U4	TPS2553DBVR	ESP32-CAM power gate (adjustable current limit).
U5	SN74LVC2G66DCTR	CAM-presence-gated UART1 isolation switch.
Q1	DN2302S	N-MOSFET low-side buzzer driver.
LS1	KPEG242-5V	Buzzer (5 V).
D5	XL-2012SURC	Red 0805 alarm LED.
D1	SMF5.0CA	VBUS input TVS clamp.
F1	Littelfuse 1206L075/16V	PTC resettable fuse (input over-current).
FB1	HCB1608KF-600T20	60 Ω ferrite bead (input EMI filter).
J2	KH-MINI-SMT-5P (Mini-USB B)	5 V input + native-USB data.
J3	ESP32-CAM (AI-Thinker)	Vision-module mezzanine connector.
J1 / J7	Pin headers	ESP32-C3 / ESP32-CAM programming & debug.

7.2 Power architecture

The board receives 5 V (VBUS) from a Mini-USB connector (J2). The input chain is: VBUS $\rightarrow$ **F1** (PTC resettable fuse; whole-board over-current protection) $\rightarrow$ **D1** (input TVS clamp) with **C1** bulk capacitor $\rightarrow$ **FB1** (ferrite bead) $\rightarrow$ the **5 V rail**. The 5 V rail then feeds, *in parallel*, the **U1** (AP2112K-3.3) LDO and the **U4** (TPS2553) CAM power gate, as well as the buzzer (LS1) and the PIR supply.

The **3.3 V rail** (U1 output) feeds **only the ESP32-C3 (U3) and the U5 switch** plus logic pull-ups. The buzzer and PIR run directly on the **5 V rail**. The ESP32-CAM runs on **CAM_5V** — the gated output of U4 — together with its own internal **CAM_3V3** regulator output; the on-board LDO does not power the CAM. ESD protection on the USB data lines is provided by U2 (USBL6-2P6) with series 22 Ω resistors; input-line TVS clamping uses SMF5.0CA devices. U4 does *not* distribute power board-wide — whole-board USB over-current protection is the job of F1.

7.3 Camera power gate (U4) and UART isolation switch (U5)

U4 (TPS2553) — ESP32-CAM power gate only. U4 is an adjustable current-limited power switch dedicated to the camera: input is the 5 V rail, output is **CAM_5V**. It is enabled by **CAM_EN** from the C3 (IO7, through R9), its current limit is set by R13 (21.5 k Ω), and its **FAULT** output feeds back to the C3 on IO3. An R11 (100 k Ω) pulldown holds the gate **OFF by default** during C3 boot, so the camera is powered only when the controller explicitly enables it. U4 does not feed the 5 V rail, the LDO, or the buzzer.

U5 (SN74LVC2G66) — automatic CAM-presence-gated UART isolation switch. U5 is a two-channel analog switch placed in series on the C3 $\leftrightarrow$ CAM UART1 link (both directions of **CAM_TXD/CAM_RXD**). Its control pins are hard-tied to the camera’s own **CAM_3V3** output, so the link *auto-connects* only when a powered CAM module is seated and *auto-disconnects* when

the CAM is absent or off. U5 is **not** a USB-to-UART programming multiplexer and is not host/GPIO-selectable.

7.4 Analytical power budget

Because no hardware has been built, the budget in Table 7 is an analytical estimate used to size the input supply, the F1 PTC fuse, and the U4 current limit, and to determine which USB source the deployment requires. The 3.3 V loads are referred to the 5 V input on the assumption that the AP2112K is a linear LDO (input current $\approx$ output current), so the per-rail currents can be summed into a single 5 V-input figure.

Table 7: Estimated application-node power budget.

Load	Voltage (V)	Current (mA)	Power (mW)
ESP32-C3 active	3.3	80	264
ESP32-CAM peak (Wi-Fi + camera on, on CAM_5V)	5.0	500	2500
Alarm LED (D5), $\approx (3.3 - 2.0)/330$	3.3	4	13
Buzzer (5 V rail through MOSFET; alarm state only)	5.0	40	200
Door contact / PIR leakage	3.3	2	7
Total (peak: CAM + alarm active)	—	626	2984

The dominant load is the ESP32-CAM during a Wi-Fi + capture burst, drawn on the gated CAM_5V branch. With the camera enabled, the estimated peak (≈ 626 mA) exceeds a basic 500 mA USB port, so the deployment supply must provide headroom: a 5 V/1 A (or better) source is recommended, and the F1 PTC fuse (750 mA hold / 1.5 A trip) and the U4 current limit (set by R13) must be coordinated with this peak (see §7.6). Because the camera is power-gated by U4 and held OFF by default (R11), the *monitoring quiescent* draw with the CAM disabled (C3 + alarm LED + sensors, $\approx 80 + 4 + 2$ mA) is only about 90 mA — well within a 500 mA USB port; sounding the buzzer in the ALARM state adds roughly 40 mA. These magnitudes are analytical estimates to be confirmed by bench measurement once hardware exists.

7.5 Output drive

The buzzer is switched by Q1, a DN2302S N-channel MOSFET. The ESP32-C3 GPIO (IO10) drives the Q1 gate through an R12 (100 Ω) series resistor, with an R14 (10 k Ω) gate pulldown to keep the buzzer off during reset/boot; the gate threshold and on-resistance must be checked against the buzzer current in the final schematic review. The alarm LED (D5) is driven from IO6 through an R15 (330 Ω) series resistor sized for the desired LED current and forward voltage.

7.6 Component value rationale

The load-bearing component values are justified below; values inferred from the net topology rather than an explicit annotation are flagged for confirmation against the Altium source.

- **Alarm-LED resistor R15 (330 Ω).** With a red LED forward voltage $V_f \approx 2.0$ V driven from the 3.3 V output IO6,

$$I_{LED} = \frac{3.3 - V_f}{R_{15}} = \frac{3.3 - 2.0}{330} \approx 3.9 \text{ mA},$$

a safe, clearly visible indicator current within the GPIO drive capability (the ≈ 4 mA used in the power budget).

- **Buzzer MOSFET Q1 (DN2302S) gate drive.** The DN2302S is a logic-level N-channel device ($V_{GS(th)} \approx 1.0\text{--}1.5$ V), so the 3.3 V drive from IO10 fully enhances it; at the ≈ 40 mA buzzer current its $R_{DS(on)}$ drop is negligible. R12 (100 Ω) limits gate inrush and R14 (10 k Ω) holds the gate low (buzzer off) while IO10 is high-impedance during reset/boot.
- **CAM power-gate limit, U4 (TPS2553) with R13 (21.5 k Ω).** The TPS2553 programmable limit follows the datasheet $I_{OS}\text{--}R_{ILIM}$ relationship over a 0.075–1.7 A range; R13 sits near the low end of the resistor range, programming a limit in the upper part of that span (order of ≈ 1.5 A — the exact threshold to be read from the TPS2553 datasheet equation for $R_{ILIM} = 21.5$ k Ω). This is above the camera’s ≈ 500 mA peak, so normal operation is not throttled, while it still caps a CAM fault. Because this limit is close to the F1 PTC trip current (≈ 1.5 A), the two must be coordinated so the gate limits before the board fuse trips — a point to confirm in review.
- **Sensor input conditioning.** The door (J4) and PIR (J5) inputs use an RC + clamp network to bound the GPIO voltage and pre-filter contact bounce / EMI, complemented by an ≈ 50 ms firmware debounce (§8). The PIR is 5 V-powered, so its output is made 3.3 V-compatible to IO5 by the on-board clamp/divider; the exact RC constant and divider ratio are to be confirmed against the schematic.

7.7 Schematic preview

The schematic is partitioned into the functional blocks in Figures 2–4: power input and regulation, the ESP32-C3 controller, the ESP32-CAM interface, the sensor inputs, and the alarm outputs. (The UART isolation switch U5 is detailed in the previous subsection.)

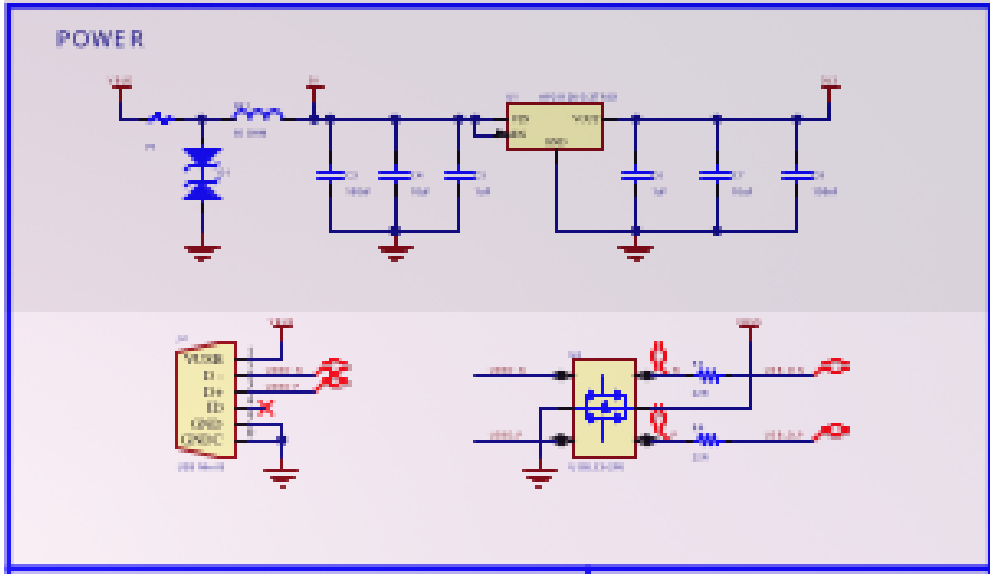


Figure 2: Power input, protection, and regulation: USB $\rightarrow$ F1 (PTC) $\rightarrow$ D1 (TVS) $\rightarrow$ FB1 $\rightarrow$ 5 V rail $\rightarrow$ U1 (AP2112K) 3.3 V LDO, with U4 (TPS2553) gating CAM_5V.

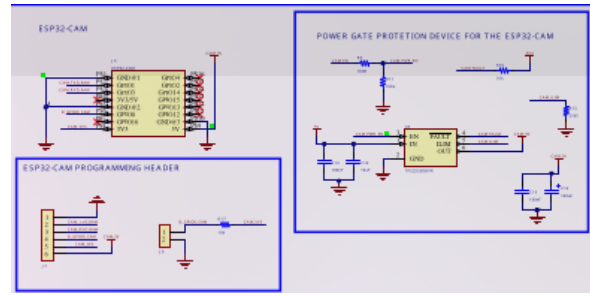
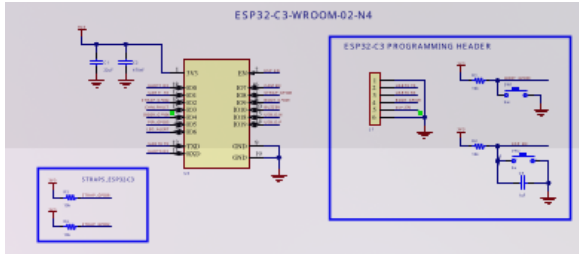


Figure 3: ESP32-C3 controller block (left) and ESP32-CAM interface block, including the U5 UART-isolation switch (right).

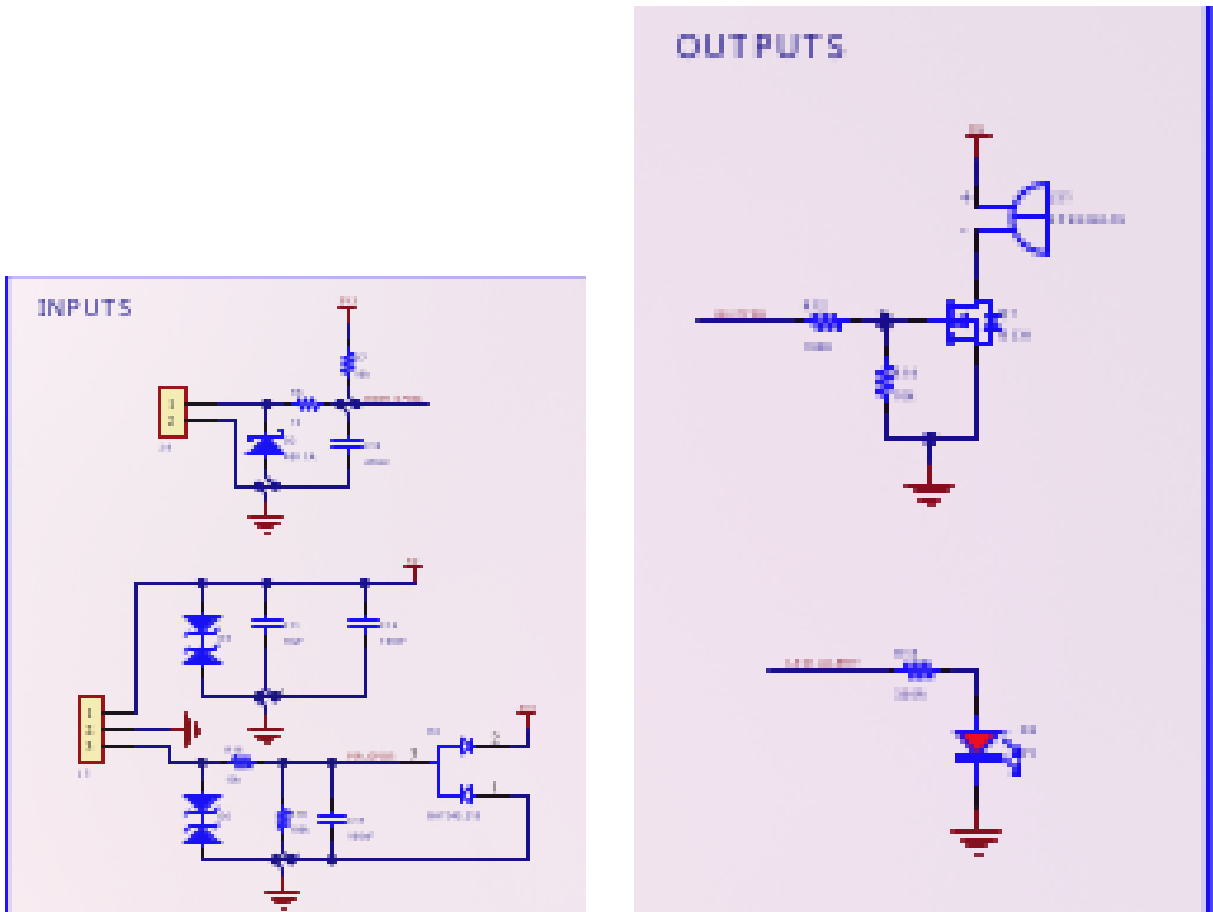


Figure 4: Sensor input conditioning (door / PIR, left) and alarm output drivers (LED and MOSFET-driven buzzer, right).

7.8 PCB design review notes

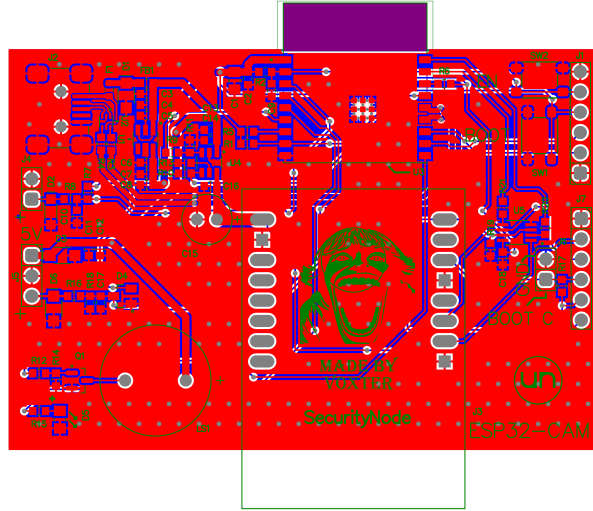


Figure 5: PCB layout preview (2-D top view). 3-D top/bottom renders are available under `assets/`.

Before fabrication, the PCB layout should be reviewed against the following checklist:

- Keep the UART1 traces between the ESP32-C3 and ESP32-CAM short and matched.
- Place decoupling capacitors close to each IC power pin.
- Provide a solid ground plane under the microcontroller and camera module.
- Route the buzzer load away from sensitive sensor traces to reduce switching noise.
- Confirm the native-USB data path ($J2 \rightarrow U2 \text{ USBLC6 ESD} \rightarrow R3/R4 \text{ } 22 \Omega \text{ series} \rightarrow IO18/IO19$) has the required ESD protection and matched series resistors.
- Verify the CAM power-gate enable/fault path: `CAM_EN` ($IO7 \rightarrow R9 \rightarrow U4 \text{ EN}$), the default-OFF pulldown ($R11$), the ILIM resistor ($R13$), and `FAULT` back to $IO3$.
- Verify the $U5$ series-isolation switch on the UART1 link is gated by `CAM_3V3` (auto presence-gating), not routed as a programmable mux.

8 State machine

The application behavior is modelled as a finite-state machine. A new state, **INCONCLUSIVE**, replaces the previous “CLEAR” state: rather than asserting that a visible (or absent) face is *safe*, the node only records that it found *no positive evidence of a covered face* — a covered face is the sole condition that escalates to **ALARM** through the vision path. This is a deliberate semantic distinction; to make it behavioural as well, repeated **INCONCLUSIVE** results within a short window may be counted toward escalation (a configurable threshold), so a stream of unreadable captures does not silently re-arm forever. The **ARMED** state maintains a `cam_available` flag, updated by periodic status queries, that determines whether the node can enter the vision-verification path or must fall back to basic-alarm behavior.

8.1 State definitions

- **INIT** — System startup, self-test, GPIO/UART initialization, and initial CAM presence check.
- **ARMED** — Normal monitoring mode. LED slow-blinks, sensors are active, and the node periodically queries the CAM status to set `cam_available`.
- **TRIGGERED** — A sensor event has been detected. The LED fast-blinks. If `cam_available` is true, the node proceeds to visual verification; otherwise it enters the ALARM state as a basic alarm.
- **VISION_REQUEST** — The node sends `CMD_CAPTURE` to the ESP32-CAM and waits for an ACK.
- **VISION_CHECK** — The node waits for the analysis result while maintaining the fast-blink indication. A 5 s timeout is enforced.
- **INCONCLUSIVE** — The CAM reports a visible face, no face, or another non-threat condition. The event is logged and the node returns to ARMED without sounding the alarm.
- **ALARM** — A suspicious condition, communication failure, or CAM-unavailable condition has occurred. The LED is solid and the buzzer is ON until the node is disarmed or the alarm timeout expires.

8.2 State-transition diagram

Figure 6 shows the updated state machine.

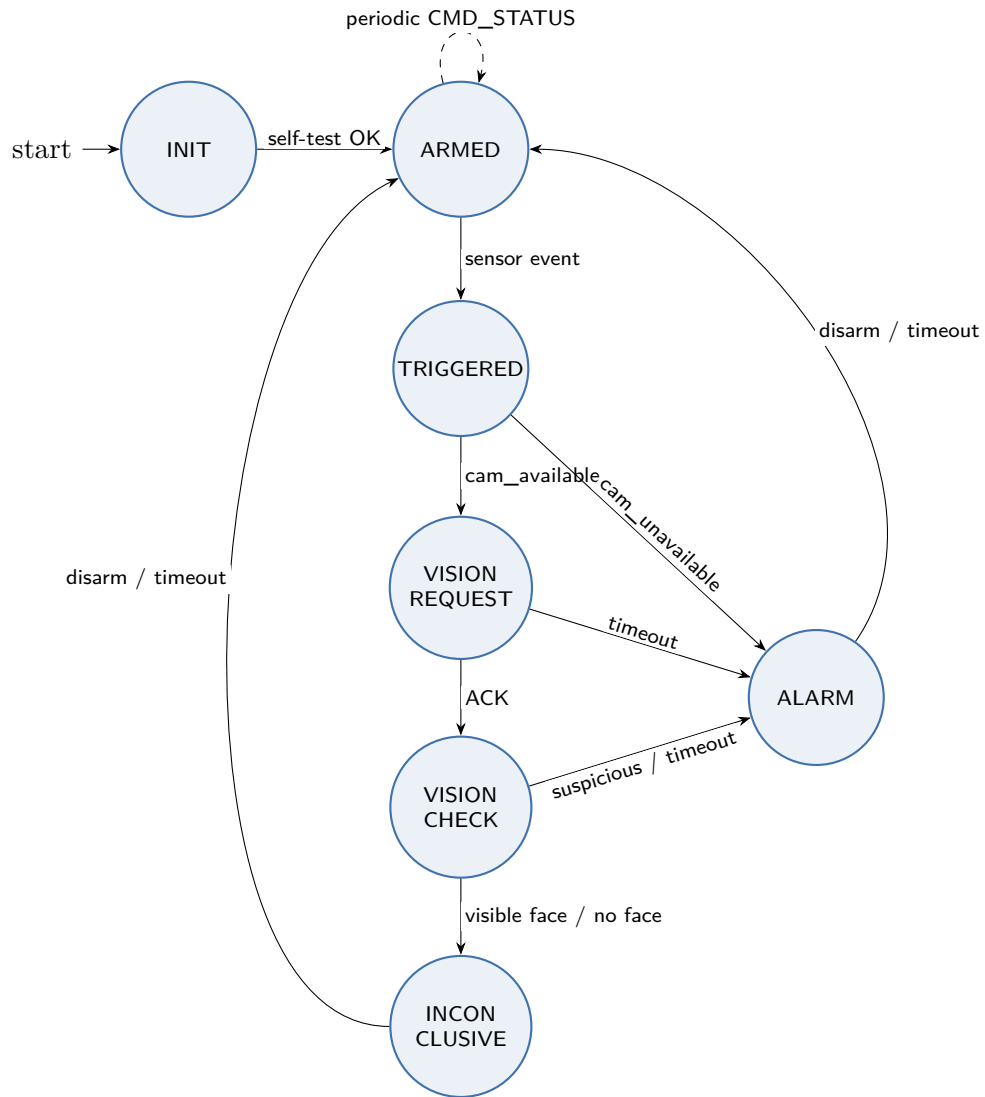


Figure 6: Application Node finite-state machine with camera-availability flag and INCONCLUSIVE state.

8.3 State behavior table

Table 8: Application Node state behavior.

State	Behavior
INIT	Power-on self-test, initialize GPIO/UART, test LED/buzzer, check CAM presence.
ARMED	LED slow blink; sensors active; periodic CAM status query updates <code>cam_available</code> .
TRIGGERED	Sensor event detected; LED fast blink; branch based on <code>cam_available</code> .
VISION_REQUEST	Send <code>CMD_CAPTURE</code> ; wait for <code>ACK</code> with 500 ms timeout and retry logic.
VISION_CHECK	Maintain fast blink; wait for <code>RSP_RESULT</code> ; enforce 5 s timeout.
INCONCLUSIVE	Non-threat result (visible face / no face); log event; return to <code>ARMED</code> .
ALARM	LED solid, buzzer ON; log event; wait for disarm or timeout.

9 UART protocol interface

The application node is the UART master for the ESP32-CAM module. Commands and responses use the binary frame defined in `docs/firmware/cam_protocol.md`, which is the authoritative source for the frame layout and the command/response IDs reproduced here.

9.1 Frame format

HEADER	LENGTH	CMD_ID	PAYLOAD	CHECKSUM
0xAA	N	1 B	N bytes	XOR

The checksum is the XOR of all bytes from `LENGTH` through the end of the payload.

Worked example. A `CMD_CAPTURE` request (no payload) is the four bytes `0xAA 0x01 0x01 0x00`: header `0xAA`, `LENGTH 0x01` (`CMD_ID` only), `CMD_ID 0x01`, and checksum $0x01 \oplus 0x01 = 0x00$. A one-byte `RSP_RESULT` carrying “suspicious” (`0x01`) is `0xAA 0x02 0x82 0x01 0x81`, where $0x02 \oplus 0x82 \oplus 0x01 = 0x81$.

9.2 Command set

Table 9: Application Node command set to the ESP32-CAM.

ID	Name	Purpose
0x01	<code>CMD_CAPTURE</code>	Request image capture and face-coverage analysis.
0x02	<code>CMD_STATUS</code>	Request current module status (updates <code>cam_available</code>).
0x03	<code>CMD_RESET</code>	Request a soft reset of the vision module.

9.3 Response handling

Table 10: Application Node response handling.

ID	Name	Action
0x81	RSP_ACK	Validate echoed command; stop retry timer.
0x82	RSP_RESULT	Evaluate result code and drive state transitions.
0x83	RSP_STATUS	Update the <code>cam_available</code> health flag.
0x84	RSP_ERROR	Log error; retry command or fall back to basic alarm.

9.4 Timeout and retry rules

Table 11: UART timeout and retry policy.

Operation	Timeout	Retries	Action on failure
ACK after command	500 ms	3	Mark CAM unresponsive; <code>cam_available=false</code> ; basic alarm.
RESULT after capture	5 s	—	Treat as suspicious; activate ALARM.
STATUS response	500 ms	2	<code>cam_available=false</code> until next successful query.

Retry loop and the disarm input. The retry counts in the timeout table are applied *inside* the VISION_REQUEST entry action: the node re-issues CMD_CAPTURE up to three times on a missing ACK before the single “timeout” transition to ALARM in Figure 6 fires (the diagram omits the retry self-loop for clarity). The **disarm** event on the return edges is not a dedicated hardware button — the board exposes only BOOT/EN — but a disarm command received on the USB-CDC / UART0 console, or expiry of the alarm timeout.

10 Firmware design sketch

The listing below is a design skeleton for the main loop. It emphasizes state updates and camera-availability handling rather than production code details.

```
void loop() {
    sensors_read(&door_state, &pir_state);

    /* Update camera health flag periodically while armed. */
    if (fsm_state == ARMED && status_query_due()) {
        cam_send_command(CMD_STATUS, NULL, 0);
    }

    fsm_update(door_state, pir_state, cam_result, cam_available);

    if (fsm_state == VISION_REQUEST && !cam_request_pending) {
        cam_send_command(CMD_CAPTURE, NULL, 0);
    }
}
```

```

    cam_request_pending = true;
    timeout_start(CAM_ACK_TIMEOUT_MS);
}

cam_process_incoming(); /* parse ACK / RESULT / STATUS / ERROR */
alarm_update(fsm_state); /* drive LED and buzzer */
debug_console_tick(); /* handle USB CLI */
}

```

11 Additional design considerations

11.1 Mechanical and environmental

The board is intended for fixed indoor installation near a monitored door. The ESP32-CAM mounts on the J3 mezzanine header with its lens facing the entry, so the enclosure must give the camera a clear field of view and keep the WROOM and ESP32-CAM PCB antennas over an antenna keep-out (no ground pour) for adequate Wi-Fi range. The sensor and USB connectors must remain accessible, and operation is assumed at normal indoor temperatures; no extended-temperature or outdoor rating is claimed at this stage.

11.2 Security considerations

As a physical-security device the design is intentionally *fail-secure*: loss of the camera or of CAM communication drives the system to ALARM rather than silently disarming (§8). Known, accepted design-stage limitations are that the USB-CDC / UART0 console is unauthenticated and the C3↔CAM UART link is unencrypted — acceptable for a wired, physically protected board, but to be revisited before any networked deployment. Loss of power and physical tampering remain residual threats; a future revision could add a tamper input and/or a small backup supply.

12 Theoretical verification plan

Because no physical prototype is available, verification is limited to analytical, inspection, and simulation-based activities.

12.1 Expected behavior and design targets

Table 12: Design targets for key behaviors.

Metric	Target	Rationale
Sensor-to-alarm latency	< 500 ms	Fast enough to deter an intruder.
ACK timeout	500 ms	Short enough to detect a stalled CAM.
Capture timeout	5 s	Allows image capture and inference without excessive delay.
Retry attempts	3	Balance robustness against UART noise with responsiveness.
Peak 5 V input current (CAM on)	≈ 626 mA	Requires a 5 V/1 A source; bounds F1 and the U4 limit.
Quiescent current (CAM gated off)	≈ 90 mA	Monitoring draw stays within a 500 mA USB port.

12.2 Analytical verification

- Confirm that every requirement in Table 2 is addressed by at least one design section or artifact.
- Walk through the state machine for all combinations of sensor events and `cam_available` values to ensure no undefined transitions.
- Verify the checksum algorithm for every command and response example in the protocol.
- Check the power budget in Table 7 against the regulator and current-limiter limits.

12.3 Schematic and PCB verification

- Inspect the Altium schematic against the pinout in Table 4.
- Confirm that each GPIO has the correct direction, pull, and protection.
- Review the PCB layout against the checklist in §7.
- Compare the `assets/Schematic Blocks/` previews with the final schematic before release.

12.4 Excluded verification activities

The following activities are deferred until a prototype is built:

- Oscilloscope capture of sensor and UART signals.
- Thermal imaging of the ESP32-C3, ESP32-CAM, and power components.
- Measured power consumption under idle, alarm, and capture conditions.
- PCB bring-up logs and hardware-in-the-loop tests.

13 References

- [1] Espressif Systems, *ESP32-C3-WROOM-02 Datasheet*.
- [2] AI-Thinker, *ESP32-CAM Module*; OmniVision, *OV2640 Camera Sensor Datasheet*.
- [3] Diodes Incorporated, *AP2112 600mA CMOS LDO Datasheet*.
- [4] Texas Instruments, *TPS2553 Adjustable Current-Limited Power-Distribution Switch Datasheet*.
- [5] Texas Instruments, *SN74LVC2G66 Dual Analog Switch Datasheet*.
- [6] STMicroelectronics, *USBLC6-2 Very-Low-Capacitance ESD Protection Datasheet*.
- [7] Doeshare, *DN2302S N-Channel MOSFET Datasheet*; TDSEMIC, *SMF5.0CA TVS Datasheet*.
- [8] Littelfuse, *1206L Series PTC Resettable Fuse Datasheet*.
- [9] Espressif Systems, *ESP-DL on-device deep-learning library*; Google, *TensorFlow Lite for Microcontrollers*.
- [10] SecurityNode project artifacts: `docs/firmware/cam_protocol.md` (UART protocol), `docs/application_node/DESIGN-AUDIT.md` (schematic-vs-docs audit), `assets/SecurityNode.BomDoc` (bill of materials).

14 Summary

This application note presented the design of the SecurityNode controller/application node. Key design decisions include:

- Replacing the unsafe “CLEAR” state with an **INCONCLUSIVE** state for non-threat vision results.
- Introducing a `cam_available` flag in the ARMED state so the node can gracefully fall back to a basic alarm when the ESP32-CAM is unavailable.
- Splitting the event-handling path into TRIGGERED, VISION_REQUEST, and VISION_CHECK to separate event detection, command issuance, and result waiting.
- Defining requirements, a traceability matrix, and a theoretical verification plan suitable for a design-stage review.

The node remains designed to work as a full vision-assisted alarm when the CAM is present and as a conventional sensor-driven alarm when it is not. Once hardware is available, the theoretical plan can be extended with bench measurements and bring-up results.